

SKILL-02

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

```
228 strace ./fork_example
229 strace -o trace.txt ./fork_example
230 cat trace.txt
231 cat /proc/$$/status
232 cat /proc/<PID>/status
233 gcc fork_example.c -o fork_example
234 ./fork_example
235 ps -ef
236 pstree -p
237 top
238 strace ./fork_example
239 cat /proc/$$/status
240 nano my_shell.c
241 gcc my_shell.c -o my_shell
242 nano my_shell.c
243 gcc my_shell.c -o my_shell
244 cat my_shell.c
245 nano input_handler.c
246 hello
247 date
248 help
249 helloo
250 cat input_handler.c
251 ./input_handler
252 nano history_shell.c
253 hello
254 nano history_shell.c
255 history
root@SRIVALLI:~# !5
passwd srivalli_osp
passwd: user 'srivalli_osp' does not exist
root@SRIVALLI:~# |
```

Report:

In this task, I implemented command history functionality to store user-entered commands. Escape sequences were used for formatted output. Previous and next command navigation was tested, and the input buffer was updated correctly when recalling commands. Different commands were entered to verify that the history and recall features worked properly. Command history was stored successfully, and previous/next command navigation and recall functionality worked as expected.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

```
Unpacking libsource-highlight-common (3.1.9-4.3build1) ...
Selecting previously unselected package libsource-highlight4t64:amd64.
Preparing to unpack .../5-libsource-highlight4t64_3.1.9-4.3build1_amd64.deb
...
Unpacking libsource-highlight4t64:amd64 (3.1.9-4.3build1) ...
Selecting previously unselected package gdb.
Preparing to unpack .../6-gdb_15.1-1ubuntu1~24.04.1_amd64.deb ...
Unpacking gdb (15.1-1ubuntu1~24.04.1) ...
Selecting previously unselected package libc6-dbg:amd64.
Preparing to unpack .../7-libc6-dbg_2.39-0ubuntu8.8_amd64.deb ...
Unpacking libc6-dbg:amd64 (2.39-0ubuntu8.8) ...
Selecting previously unselected package valgrind.
Preparing to unpack .../8-valgrind_1%3a3.22.0-0ubuntu3_amd64.deb ...
Unpacking valgrind (1:3.22.0-0ubuntu3) ...
Setting up libdebuginfod-common (0.190-1.1ubuntu0.1) ...
Setting up libsource-highlight-common (3.1.9-4.3build1) ...
Setting up libc6-dbg:amd64 (2.39-0ubuntu8.8) ...
Setting up libipt2 (2.0.6-1build1) ...
Setting up libbabeltrace1:amd64 (1.5.11-3build3) ...
Setting up valgrind (1:3.22.0-0ubuntu3) ...
Setting up libdebuginfod1t64:amd64 (0.190-1.1ubuntu0.1) ...
Setting up libsource-highlight4t64:amd64 (3.1.9-4.3build1) ...
Setting up gdb (15.1-1ubuntu1~24.04.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.8) ...
root@SRIVALLI:~# valgrind ./dynamic_memory
valgrind: ./dynamic_memory: No such file or directory
root@SRIVALLI:~# valgrind --leak-check=full ./dynamic_memory
valgrind: ./dynamic_memory: No such file or directory
root@SRIVALLI:~# valgrind --leak-check=full ./dynamic_memory
valgrind: ./dynamic_memory: No such file or directory
root@SRIVALLI:~# cat dynamic_memory.c
gcc dynamic_memory.c -o dynamic_memory

root@SRIVALLI:~# nano dynamic_memory.c
root@SRIVALLI:~# ./dynamic_memory
-bash: ./dynamic_memory: No such file or directory
root@SRIVALLI:~#
```

Report:

In this task, I used dynamic memory allocation to create buffers and resize arrays during program execution. Buffer overflow was avoided by allocating sufficient memory and validating input sizes. Linked list operations were performed using dynamically allocated memory. All allocated memory was released correctly using `free()`, and Valgrind was used to verify that there were no memory leaks. Dynamic memory allocation, array resizing, and linked list management were implemented successfully. Memory usage was verified using Valgrind, and no memory leaks were detected.

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

```
Preparing to unpack .../6-gdb_15.1-1ubuntu1~24.04.1_amd64.deb ...
Unpacking gdb (15.1-1ubuntu1~24.04.1) ...
Selecting previously unselected package libc6-dbg:amd64.
Preparing to unpack .../7-libc6-dbg_2.39-0ubuntu8.8_amd64.deb ...
Unpacking libc6-dbg:amd64 (2.39-0ubuntu8.8) ...
Selecting previously unselected package valgrind.
Preparing to unpack .../8-valgrind_1%3a3.22.0-0ubuntu3_amd64.deb .
Unpacking valgrind (1:3.22.0-0ubuntu3) ...
Setting up libdebuginfod-common (0.190-1.1ubuntu0.1) ...
Setting up libsource-highlight-common (3.1.9-4.3build1) ...
Setting up libc6-dbg:amd64 (2.39-0ubuntu8.8) ...
Setting up libipt2 (2.0.6-1build1) ...
Setting up libbabeltrace1:amd64 (1.5.11-3build3) ...
Setting up valgrind (1:3.22.0-0ubuntu3) ...
Setting up libdebuginfod1t64:amd64 (0.190-1.1ubuntu0.1) ...
Setting up libsource-highlight4t64:amd64 (3.1.9-4.3build1) ...
Setting up gdb (15.1-1ubuntu1~24.04.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.8) ...
root@SRIVALLI:~# valgrind ./dynamic_memory
valgrind: ./dynamic_memory: No such file or directory
root@SRIVALLI:~# valgrind --leak-check=full ./dynamic_memory
valgrind: ./dynamic_memory: No such file or directory
root@SRIVALLI:~# valgrind --leak-check=full ./dynamic_memory
valgrind: ./dynamic_memory: No such file or directory
root@SRIVALLI:~# cat dynamic_memory.c
gcc dynamic_memory.c -o dynamic_memory

root@SRIVALLI:~# nano dynamic_memory.c
root@SRIVALLI:~# ./dynamic_memory
-bash: ./dynamic_memory: No such file or directory
root@SRIVALLI:~# nano tokenizer.c
root@SRIVALLI:~# gcc tokenizer.c -o tokenizer
```

Report:

In this task, I implemented tokenization of user input by splitting commands into individual tokens using delimiters such as spaces. Whitespace handling was performed to ensure correct parsing of commands. Token structures were created and validated to verify the correctness of the input stream. Different commands were tested to observe the generated tokens and debug the parsing output. User input was successfully split into tokens, delimiters and whitespace were handled correctly, and the token stream was validated successfully.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
root@SRIVALLI:~# nano parser.c
root@SRIVALLI:~# ./parser
-bash: ./parser: No such file or directory
root@SRIVALLI:~# ls -l
total 96
drwxr-xr-x 3 root root 4096 Aug 10 11:27 OSSP
drwxr-xr-x 6 root root 4096 Aug 20 05:49 SkillWeek1
-rwxr-xr-x 1 root root 16488 Aug 17 05:11 command_executor
-rw-r--r-- 1 root root 1050 Aug 17 05:10 command_executor.c
-rw-r--r-- 1 root root 1050 Aug 17 05:10 command_executor.c.
-rw-r--r-- 1 root root 41 Aug 22 06:26 dynamic_memory.c
-rwxr-xr-x 1 root root 16224 Aug 14 05:05 fork_process
-rw-r--r-- 1 root root 850 Aug 14 05:05 fork_process.c
-rw-r--r-- 1 root root 105 Aug 22 05:58 history_shell.c
-rw----- 1 root root 3 Aug 4 17:12 nano.443.save
-rw-r--r-- 1 root root 24 Aug 22 06:35 parser.c
-rwxr-xr-x 1 root root 16096 Aug 14 09:54 process_state
-rw-r--r-- 1 root root 278 Aug 14 09:54 process_state.c
-rw-r--r-- 1 root root 30 Aug 22 06:29 tokenizer.c
root@SRIVALLI:~# pwd
/root
root@SRIVALLI:~# mkdir test
root@SRIVALLI:~# &&&
-bash: syntax error near unexpected token `&&'
root@SRIVALLI:~# Enter
Enter: command not found
root@SRIVALLI:~# cat parser.c
gcc parser.c -o parser
```

Report:

In this task, I designed a parser to analyze user commands and validate their syntax. The parser was used to identify valid and invalid commands, detect errors, and handle empty inputs. Different commands were tested to observe the parsing process and verify the generated execution structure. This task helped me understand how a shell checks and processes user commands before execution. The parser successfully validated command syntax, detected invalid and empty commands, and generated the required execution structures for command processing.